Kshitiz Gupta

INTERMEDIATE SQL ASSIGNMENT 2

1. Perform following queries:

a. Find the top 3 customers who have spent the most on orders using a CTE.

Query:-

```
-- first i have to create CTE
WITH spent AS (
    SELECT c.customer_id,
    c.name,
    SUM(o.total_amount) AS total_amount
    FROM customers c
    JOIN orders o
    ON c.customer_id=o.customer_id
    GROUP BY c.customer_id,c.name
)
SELECT *
FROM spent
ORDER BY total_amount
DESC LIMIT 3;
```

Query 1

Limit to 1000 rows

```sql
-- first i have to create CTE
WITH spent AS (
    SELECT c.customer_id,
    c.name,
    SUM(o.total_amount) AS total_amount
    FROM customers c
    JOIN orders o
    ON c.customer_id=o.customer_id
    GROUP BY c.customer_id,c.name
)
SELECT *
FROM spent
ORDER BY total_amount
DESC LIMIT 3;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	name	total_amount
9	Karan Patel	900.00
1	Amit Sharma	800.00
2	Riya Singh	600.00

Result 3

Output

Action Output

#	Time	Action	Message
1	13:04:24	WITH spent AS (SELECT c.customer_id, c.name, SUM(o.total_amount) AS total_amount FROM c...	3 row(s) returned
2	13:04:34	WITH spent AS (SELECT c.customer_id, c.name, SUM(o.total_amount) AS total_amount FROM c...	3 row(s) returned

b. Retrieve all orders where the total amount exceeds 500, converting the amount to an integer. Using CAST

Query:-

```
-- converting decimal to signed integer
SELECT *
FROM orders
WHERE CAST(total_amount AS SIGNED) > 500;
```

```
-- converting decimal to signed integer
SELECT *
FROM orders
WHERE CAST(total_amount AS SIGNED) > 500;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

order_id	customer_id	product_id	quantity	order_date	status	payment_method	total_amount
1	1	1	1	2024-01-01	Success	Credit	800.00
2	2	2	2	2024-01-02	Pending	Debit	600.00
6	6	6	1	2024-01-06	Success	Debit	600.00
9	9	9	1	2024-01-09	Success	Credit	900.00
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

orders 8

Output

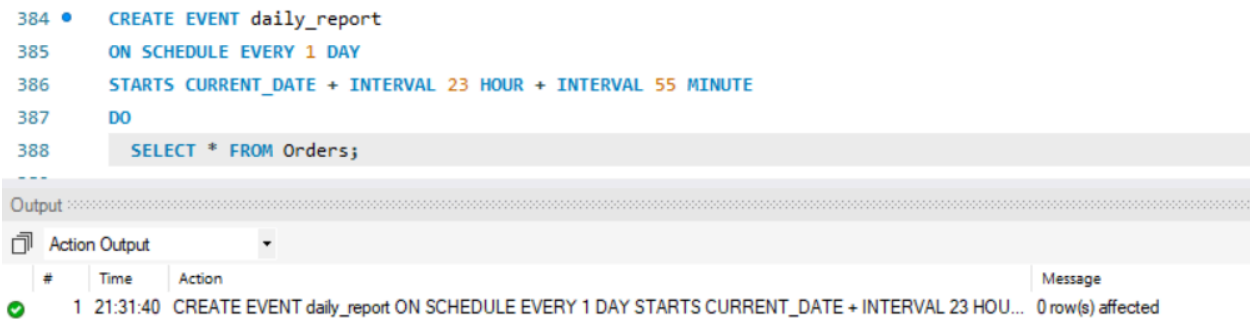
Action Output

#	Time	Action	Message
1	15:25:00	SELECT * FROM orders WHERE CAST(total_amount AS SIGNED) > 500 LIMIT 0, 1000	4 row(s) returned

c. Write a CRON job to generate daily sales reports at 11:55 PM

Query:-

```
CREATE EVENT daily_report
ON SCHEDULE EVERY 1 DAY
STARTS CURRENT_DATE + INTERVAL 23 HOUR + INTERVAL 55
 MINUTE
DO
  SELECT * FROM Orders;
```



CRON Job

55 23 * * * /path/to/script.sh

Explanation:

This cron job schedules a script to run every day at 11:55 PM to generate sales reports.

A CRON job is a Linux-based command used for scheduling tasks to run automatically at specified times.

d. Write a transaction to insert a new customer and an order. If the order fails, roll back the customer creation.

Query:-

```
START TRANSACTION;

-- Insert new customer
INSERT INTO Customers (name, email, mobile, age)
VALUES ('Ravi Kumar', 'ravi@gmail.com', '9876501111', 30);

-- Insert order for that customer
INSERT INTO Orders (customer_id, product_id, quantity, order_date,
total_amount)
VALUES (LAST_INSERT_ID(), 1, 2, CURDATE(), 1600);

-- if everything is succesful
COMMIT;

-- if there is any problem
ROLLBACK;
```

```
389
390 •   START TRANSACTION;
391
392     -- Insert new customer
393 •   INSERT INTO Customers (name, email, mobile, age)
394     VALUES ('Ravi Kumar', 'ravi@gmail.com', '9876501111', 30);
395
396     -- Insert order for that customer
397 •   INSERT INTO Orders (customer_id, product_id, quantity, order_date, total_amount)
398     VALUES (LAST_INSERT_ID(), 1, 2, CURDATE(), 1600);
399
400     -- if everything is succesful
401 •   COMMIT;
402
403     -- if there is any problem
404 •   ROLLBACK;
```

Output

Action Output

#	Time	Action	Message
1	21:31:40	CREATE EVENT daily_report ON SCHEDULE EVERY 1 DAY STARTS CURRENT_DATE + INTERVAL 23 HOU...	0 row(s) affected
2	21:51:14	START TRANSACTION	0 row(s) affected
3	21:51:51	INSERT INTO Customers (name, email, mobile, age) VALUES ('Ravi Kumar', 'ravi@gmail.com', '9876501111', 30)	1 row(s) affected
4	21:52:09	INSERT INTO Orders (customer_id, product_id, quantity, order_date, total_amount) VALUES (LAST_INSERT_ID(...	1 row(s) affected
5	21:52:19	COMMIT	0 row(s) affected
6	21:52:24	ROLLBACK	0 row(s) affected

e. Rank customers based on their total spending, showing ranks even if values are tied.

Query:-

```
SELECT
    c.customer_id,
    c.name,
    SUM(o.total_amount) AS total_spent,
    RANK() OVER (ORDER BY SUM(o.total_amount) DESC) AS
rank_position
FROM Customers c
JOIN Orders o
    ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name;
```

Query 1

Limit to 1000 rows

```
SELECT
    c.customer_id,
    c.name,
    SUM(o.total_amount) AS total_spent,
    RANK() OVER (ORDER BY SUM(o.total_amount) DESC) AS rank_position
FROM Customers c
JOIN Orders o
    ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	name	total_spent	rank_position
18	Nikita Shah	500.00	6
10	Simran Kaur	300.00	8
4	Sneha Jain	210.00	9
14	Meera Nair	200.00	10
15	Suresh Kumar	160.00	11
17	Deepak Singh	150.00	12
20	Alok Pandey	135.00	13
8	Pooja Sharma	120.00	14
11	Manish Yadav	120.00	14
12	Anita Das	120.00	14
13	Rohit Roy	120.00	14
3	Rahul Verma	100.00	18
16	Priya Iyer	100.00	18
19	Raj Malhotra	90.00	20
5	Vikas Gupta	80.00	21

Result 10

Output

Action Output

#	Time	Action	Message
1	21:59:43	SELECT c.customer_id, c.name, SUM(o.total_amount) AS total_spent, RANK() OVER (ORDER BY S...	21 row(s) returned

Limit to 1000 rows

```
SELECT
    c.customer_id,
    c.name,
    SUM(o.total_amount) AS total_spent,
    RANK() OVER (ORDER BY SUM(o.total_amount) DESC) AS rank_position
FROM Customers c
JOIN Orders o
    ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	name	total_spent	rank_position
21	Ravi Kumar	1600.00	1
9	Karan Patel	900.00	2
1	Amit Sharma	800.00	3
2	Riya Singh	600.00	4
6	Neha Kapoor	600.00	4
7	Arjun Mehta	500.00	6
18	Nikita Shah	500.00	6
10	Simran Kaur	300.00	8
4	Sneha Jain	210.00	9
14	Meera Nair	200.00	10
15	Suresh Kumar	160.00	11
17	Deepak Singh	150.00	12
20	Alok Pandey	135.00	13
8	Pooja Sharma	120.00	14
11	Manish Yadav	120.00	14

Result 10

Output

Action Output

#	Time	Action	Message
1	21:59:43	SELECT c.customer_id, c.name, SUM(o.total_amount) AS total_spent, RANK() OVER (ORDER BY S...	21 row(s) returned